

SOLID PRINCIPLES

Set of principles, which must be followed to develop flexible, maintainable and Scalable Software designs.

S → Single Responsibility Principle
[One Responsibility]

O → Open/closed principle
[Extend, Don't Modify]

L → Liskov Substitution Principle
[Parent = child behaviour safe]

I → Interface Segregation principle
[small Interfaces]

D → Dependency Inversion Principle
[depend on abstractions]

Single Responsibility Principle [SRP]

* Each class should have only one Responsibility.

Violation of SRP: [Multiple Responsibilities]

C# Program

```
class BankAccount
{
    public void Deposit() {}
    public void Withdraw() {}
    public void PrintStatement() {}
}
```

Valid [separate responsibilities]

```
class BankAccount
{
    public void Deposit() {}
    public void Withdraw() {}
}
```

```
class AccountStatement
```

```
{
    public void PrintStatement() {}
}
```

(One class = One Reason to change)

Open / closed

principle [OCP]

* Software Entities like (classes, modules etc.)
Should be open for extension & closed for
Modification.

Example

Ocp violation

* New account type can
be added without changing
existing code

class InterestCalculator

{

public double calculate (String accountType)

{
if (accountType == "Savings")

return 5;

else

return 3;

}
}

Valid

abstract class Account

{
public abstract double calculateInterest();
}

class SavingsAccount : Account

{

public override double calculateInterest()

{

return 5;

}

}

class CurrentAccount : Account

{

public override double calculateInterest()

{

return 3;

}

Liskov Substitution principle (LSP)

* An object of child class must be able to replace object of the parent class without breaking the application.

Simple Words $\Rightarrow$ All base class methods must be applicable for the derived / child class.

Violation of LSP:-

```
class Account
```

```
{  
    public virtual void Withdraw (double amount)  
    {  
        Console.WriteLine ("Amount Withdraw");  
    }  
}
```

```
class Fixed Deposit Account : Account
```

```
{  
    public override void Withdraw (double amount)  
    {  
        // FD does not allow withdrawal  
        throw new NotSupportedException ("Not allowed");  
    }  
}
```

```
class program
```

```
{  
    static void Main ()
```

```
{  
    Account acc = new Fixed Deposit Account ();
```

```
    acc.Withdraw (1000); // X Runtime Error
```

```
}  
}
```

Why this violates?

* Account class $\Rightarrow$ Withdraw() will work
Promises

* Fixed Deposit Account $\Rightarrow$ breaks that
[child class]

* Parent reference cannot safely use child
Object.

* This breaks Rule of LSP [breaks substitutability]

[Base class method $\Rightarrow$ not applicable for derived]

Valid :-

```
abstract class Account
```

```
{  
    public abstract void Withdraw (double amount);  
}
```

```
class SavingsAccount : Account
```

```
{  
    public override void Withdraw (double amount)
```

```
{  
        Console.WriteLine ("Withdraw from Savings");
```

```
    }  
}
```

* Fixed Deposit Account child class should not
inherit from Account [parent class], if withdrawal
is not allowed.

Interface Segregation Principle [ISP]

Class should not be forced to implement the interfaces that it doesn't use.

Violation of ISP:

```
interface IAccount
```

```
{
```

```
    void Deposit();
```

```
    void Withdraw();
```

```
    void Loan();
```

```
}
```

// It forces all implementing classes to support Loan(), even when they don't need it.

Valid

```
interface IAccount
```

```
{
```

```
    void Deposit();
```

```
    void Withdraw();
```

```
}
```

```
interface ILoanAccount
```

```
{
```

```
    void Loan();
```

```
}
```

```
class SavingsAccount : IAccount
```

```
{
```

```
    public void Deposit() { }
```

```
    public void Withdraw() { }
```

```
}
```

Small, specific interfaces are better than larger one.

Dependency Inversion Principle [DIP]

* Depend on abstractions, not concrete / child classes.

* High level code should not depend on Low level code; both should depend on abstractions.

Violation of DIP:

```
class EmailService
```

```
{
```

```
    public void send()
```

```
    {
```

```
        Console.WriteLine("Email Sent");
```

```
    }
```

```
}
```

```
class Notification
```

```
{
```

```
    EmailService email = new EmailService();
```

```
    public void Notify()
```

```
    {
```

```
        email.Send();
```

```
    }
```

```
}
```

//direct dependency

* Notification depends directly on EmailService.

* If we need SMS (or) WhatsApp in future



code must be change

Valid

```
interface IMessageService
```

```
{
```

```
    void Send();
```

```
}
```

```
class EmailService : IMessageService
```

```
{
```

```
    public void Send()
```

```
    {
```

```
        Console.WriteLine("Email sent");
```

```
    }
```

```
}
```

```
class SMSService : IMessageService
```

```
{
```

```
    public void Send()
```

```
    {
```

```
        Console.WriteLine("SMS sent");
```

```
    }
```

```
}
```

```
class Notification
```

```
{
```

```
    IMessageService messageService;
```

```
    public Notification(IMessageService service)
```

```
    {
```

```
        messageService = service;
```

```
}
```



```

public void Notify ()
{
    messageservice.Send();
}
}

```

class program

```

{
    static void Main()
    {
        Notification n1 = new Notification (new EmailService());
        Notification n2 = new Notification (new SMSService());
        n1.Notify();
        n2.Notify();
    }
}

```

* Notification → high level module

* Email service / SMS Service ⇒ Low Level Module

* Both depends on IMessageService.

* No code change needed if new message type needed.

* Loose coupling achieved.